Object Recognition
Practical Sessions

Meysam Madadi
meysam.madadi@gmail.com

Date	Theory title	Teacher	Practical session
17-02 - 2026	Presentation and CNN basics	Sergio	Introduction to practice 1, 2 and 3
24-02 - 2026	CNN and GNN architectures	Meysam	P1 Q&A
03-03 - 2026	Object detection and segmentation	Meysam	P1 presentation
10-03 - 2026	NO CLASS		
17-03 - 2026	Recurrent models and transformers	Meysam	P2 Q&A
24-03 - 2026	NO CLASS		
31-03 - 2026	Easter holidays		
07-04 - 2026	Exams week		
14-04 - 2026	Human pose estimation	Meysam	P2 presentation
21-04 - 2026	Fairness and bias	Julio (INVITED SPEAKER)	-
28-04 - 2026	Presentation I	Sergio	-
05-05 - 2026	Presentation II	Sergio	-
12-05 - 2026	Generative models	Meysam	P3 Q&A
19-05 - 2026	Video understanding	Albert (INVITED SPEAKER)	-
26-05 - 2026	3D vision	Meysam	P3 presentation
02-06 - 2026	Exam preparation		
09-06 - 2026	Exam	Meysam	

7 sessions
3 blocks

Deliverables

1. CNN architecture design, deadline 03/03/2026 09:00
2. Fashion parsing (segmentation), deadline 14/04/2026 09:00
3. Body and clothes depth estimation, deadline 26/05/2026 09:00

Notes:

- The tasks must be done in the groups of **5** students. One student in each group must send me the names of his/her mates by the end of the week. If I do not receive this email, I will assign students to the groups randomly.
- Deliverable is a zip file including the code, report pdf and presentation slides.
- The report must fit within the **10** page limit using the font **calibri** with size **11**. Extra pages will be penalized.
- Submissions must be done through virtual campus.
- Submissions after the deadline will be penalized.
- The solutions and results must be presented by ALL the members of each group in **10 minutes** on the defined days,
- Deliverables are evaluated based on the quality of the report and presentation, and the number of tasks done. Erroneous results without proper justification won't be taken into account. Include **qualitative** images.

Deliverable 1 - CNN architecture design

- The goal is to learn how to design and train CNN networks to maximize mAP,
- The task is the multi-label classification on the **Pascal VOC 2012** dataset,
- You can use the given code in the practical session as your base code.
  - Or reimplement it with your desired library (e.g. pytorch)

Evaluation Metric

- Primary:
  - **Mean Average Precision (mAP)** (VOC-style)
- Secondary:
  - Per-class AP and confusion matrix
  - Inference time (ms/image)
  - Model size (MB)

Deliverable 1 - Tasks

1- Analyze the dataset for the distribution of the labels by:

a. Counting the objects;
b. Counting the images, that is no matter how many objects of the same label appear on the image, all counts 1;
c. Mean and variance of the area ratio (dividing object area to image area) of objects per label. You can use object bounding box to compute the area.

Deliverable 1 - Tasks

2- Train a **baseline CNN**: implement VOC 2012 data loader and encode labels as **multi-hot vectors**

- Architecture: **ResNet-18**
- Pretrained on ImageNet
- Loss: **BCEWithLogitsLoss**
- Optimizer: Adam
- Minimal augmentation (resize + normalize)

Deliverable 1 - Tasks

3- Architecture comparison: study accuracy vs efficiency trade-offs

- Same augmentation, optimizer, and training schedule as baseline
- Architectures to compare: ResNet-18, ResNet-50, MobileNetV3-Large, EfficientNet-B0

Questions:

- Does a deeper model always help?
- Which model gives best **mAP / GPU-hour**?
- Which struggles most with small objects?
- What about overfitting?

Select the best architecture and use it for the next experiments.

Deliverable 1 - Tasks

4- Transfer learning strategies:

- Train everything **from scratch**
- Freeze the pretrained backbone and only train the head
- Gradual unfreezing
- Discriminative learning rates
- Warm-up vs no warm-up

Questions:

- Analyze the convergence speed, overfitting and best strategy

Select and use the best strategy for the next experiments.

Deliverable 1 - Tasks

5- Data and loss design: analyze class frequency and improve performance via better supervision

- **Data Augmentation**
  - Horizontal flip, color jitter, random resized crop, MixUp / CutMix
- **Dataset Imbalance**
  - Weighted BCE
  - Focal Loss
- **Classification Head**
  - Linear vs 2-layer MLP
  - Dropout

Questions:

- Analyze the class frequency vs. per class AP
- Does the network converge with the same speed for all classes? Just brainstorm some mitigation ideas without any implementation or training

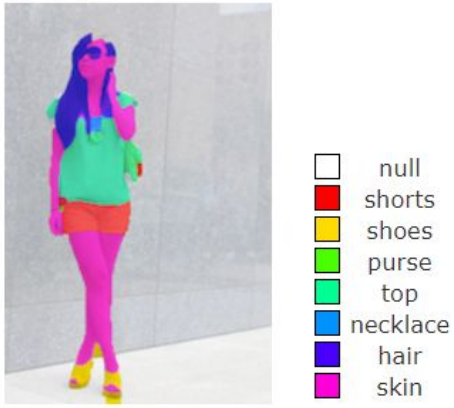
Select and use the best strategy for the next experiments.

Deliverable 1 - Tasks: Optional Bonus

6- Further optional improvements:

- Tune hyperparameters: learning rate and batch size
- Train with mixed precision
- Tune per-class threshold

Deliverable 2 - fashion parsing



Clothing parsing

What do you need to do in this task?

- Fashion semantic segmentation

On which dataset?

- Fashionpedia: https://fashionpedia.github.io/home/

Using which code base?

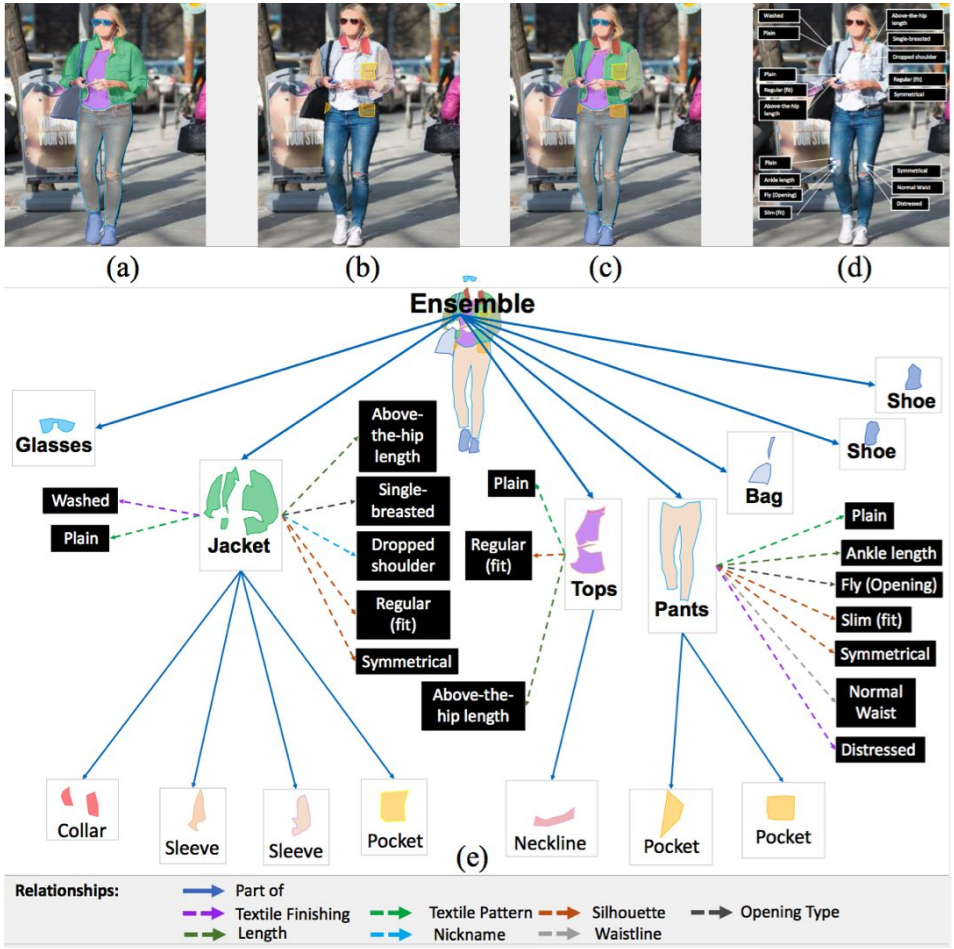
- MMSegmentation: https://github.com/open-mmlab/mmsegmentation
- Or any code you find more convenient

Older versions are more stable

Fashion parsing

Hierarchical segmentation and attribute detection

- <u>We do not consider item parts and attributes in this practice</u>,
- Only the person in the middle has been annotated.
- We use the validation set to report the results.



Deliverable 2 - Tasks

1- Analyze the dataset for the distribution of the labels by:

- Counting the number of images per label,
- Mean and variance for the pixel ratio per label:
  - (# of pixels of label j in image i) / (# of pixels in image i),
  - (# of pixels of label j in image i) / (# of **foreground** pixels in image i) ====> in this case we exclude the background

Deliverable 2 - Tasks

2- Choose three segmentation networks (justification is required) from the MMSegmentation implementations and finetune them on the fashionpedia dataset, as far as your resources allows, as following:

- Tune the learning rate and batch size,
- Apply the basic data augmentation like rotation, scaling, cropping, mosaic, cutout, mixup, etc,
- Train based on 192 vs 384 px image resolution. Is there any relationship between the resolution and accuracy of each label?

Select the best performing network and resolution and do the following:

- Discard the large and overrepresented labels (check per label metric) and finetune the network with the same hyperparameters as above. Does this make any difference to the results of the remaining labels?

Deliverable 2 - Tasks

3- Train YOLO based instance segmentation and compare the results with the previous models.

Deliverable 2 - Tasks

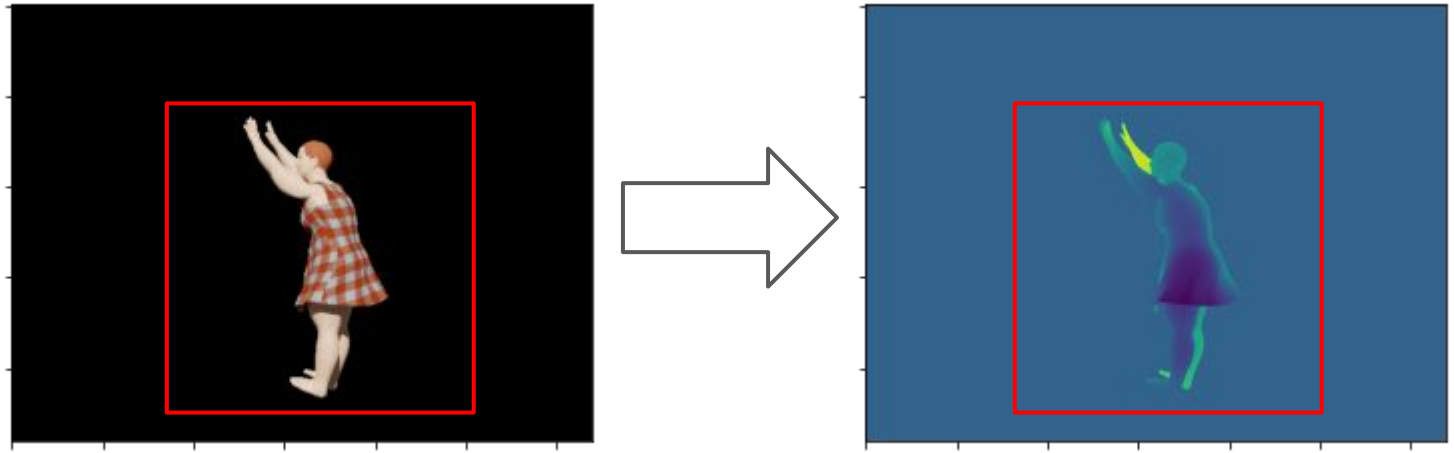
4- Without implementing any code or training, just discuss two possible ideas to improve the results. Back up your idea by referring to SOTA.

NOTES for writing the report:

- Justify your choices,
- Thoroughly discuss the results both qualitatively and quantitatively,
- Report the results using accuracy and mDice metrics excluding the background,
- Your best performing network must show at least 20% mDice.

Deliverable 3 - Body and cloth depth estimation

- Given an image of a person, cropped in the preprocessing (red rectangle), the goal is to estimate its corresponding depth image.



Deliverable 3 - Body and cloth depth estimation

- Dataset:
  - A subset of the CLOTH3D++ dataset (https://chalearnlap.cvc.uab.cat/dataset/38/description/), could be downloaded from cloth3d++_subset.zip. The first 128 folders (00001 to 00152) must be used for training, the following 16 folders (00153 to 00168) for validation and the rest for test.
  - You can play with the data, see data structure, visualize the 3D, render depth images and extract frames from the RGB videos using the given starter kit.
  - If you use colab, the content of the starter kit must be uploaded to your google drive under “cloth3d” folder.
- Preprocessing:
  - Crop and save the images such that 1) the center of the subject and cropping to be the same, 2) leave 10px margin between cropping and subject boundaries, and 3) apply square cropping. NOTE: in some frames the subject may go out ot the scene. You can ignore these frames, for instance by counting the number of pixels in the mask and thresholding. The frames must be saved as .jpg under “image” folder with naming protocol “<folder name where the video is located>_<frame number>”
  - Rendered depth must be cropped similar to RGB frame and saved as .npy (using numpy.save) under “depth” folder with the same naming protocol as RGB frames.
  - The cropped images can be resized to 256x256 before saving them. Note that resizing the depth images could be tricky especially for the boundaries due to the effect of the zero background. You can compare nearest neighbor vs bilinear interpolation for resizing.

Deliverable 3 - Body and cloth depth estimation

Metrics

- **Depth:** RMSE
- **Normals:** Mean angular error
- **Efficiency:**
  - FPS
  - GPU memory
  - Parameter count
- Support your results with **qualitative** images

Normals can be computed as the gradients of the depth image

Deliverable 3 - Body and cloth depth estimation

1- Data preprocessing and baseline training:

- Preprocess the data as stated before,
- A simple baseline code is given for depth estimation using a light weight UNET architecture,
- Explore:
  - Resolution: 256 vs 384,
  - Relevant data augmentation: shifting, horizontal flip, scaling/cropping, rotation, shearing
  - Learning rate vs batch size importance

Questions:

- What makes monocular depth ill-posed?
- Compare the results

Deliverable 3 - Body and cloth depth estimation

2- Transformer architectures for dense prediction:

- Replace UNET with one of the following encoder architectures (pretrained on ImageNet):
  - DeiT-Small ➡ simple and efficient
  - Swin-Tiny ➡ hierarchical features
  - EfficientFormer ➡ Transformer–CNN hybrid
- Implement a decoder: simple bilinear upscaling vs lightweight feature pyramid network
- Explore:
  - Patch size 8 vs 16,
  - Two transfer learning techniques that you learned in the first practice,

Questions:

- Do transformers preserve spatial detail?
- Where do they fail vs CNNs?
- What is more important in accuracy gains: transfer learning, patch size, decoder design, etc?
- Discuss architecture diagram, memory vs speed, performance vs CNN,

Deliverable 3 - Body and cloth depth estimation

3- weak supervision and geometry-aware losses:

- Weak Supervision
  - Ranking loss
  - Normal consistency loss: implement a differentiable normal map layer derived from the network output vs GT depth and apply per pixel L1 loss
- Geometry-aware
  - Scale-invariant depth loss
  - Body pose and depth multi-tasking: by the help of the starter kit, project 14 main SMPL joints to the image and use XY coordinates to train the multi-tasking model. To do so, create another decoder branch with the shape (Height, Width, 14) and apply soft-argmax on each output channel. Then, perform L1 loss between soft-argmax and GT XY coordinates. For the depth branch, use pixel-wise L1 loss between the GT and estimated depth.

Questions:

- Analyze and compare:
  - Full supervision vs weak-only vs geometry aware,
  - Failure modes: textured vs smooth areas, around edges, etc
  - Stability vs accuracy
  - Losses: convergence speed, overfitting, etc

Deliverable 3 - Body and cloth depth estimation

4- final integration: maximize the performance with what you've learned so far.